

Validação unificada de certificados —

`cert.fambrashalal.com.br/verify`

Para: equipe de Infra · **Data:** 2026-06-22 · **Conta AWS:** 767397935861 · **Região:** us-east-1 **Status:** em produção.

1. Objetivo

Servir a tela pública de validação de certificados do **GC (Gestão de Certificações / HalalSphere)** sob o **mesmo domínio** já usado pelo SysHalal — `cert.fambrashalal.com.br` — por exigência de credibilidade (organismos internacionais). Solução **aditiva**: o SysHalal (`/certificadovalidate/*`) **não foi tocado**; nenhum cutover de DNS; nenhum CloudFront novo.

Resultado:

- `cert.fambrashalal.com.br/verify/*` → GC (novo)
- `cert.fambrashalal.com.br/certificadovalidate/*` → SysHalal (inalterado)

2. Arquitetura (fluxo de request)

```
QR do certificado / navegador
| https://cert.fambrashalal.com.br/verify/{numeroCert}
▼
Route53 (A) → ALB ecohalal-fambrashalal-web (internet-facing)
| listener HTTPS:443 — TLS *.fambrashalal.com.br (wildcard, na ALB)
| └─ regra prio 10: path /verify/* → TG halalosphere-verify-web-tg
|                                     → ECS Fargate (nginx :3000)
|                                     • serve a SPA do GC (base /verify/)
|                                     • /verify/api/* → proxy server-side →
API GC
└─ ação default: /certificadovalidate/* etc → TG fambrashalal-cert-web-tg (SysHalal,
intocado)
```

Detalhe do `/verify/api` : a SPA chama `cert.fambrashalal.com.br/verify/api/...` (same-origin). O **nginx do container faz o proxy server-side** para a API do GC (`https://gestaodecertificacoes-api.ecohalal.solutions`). O navegador **nunca vê** a origem interna; e como é same-origin, **não há CORS**.

3. Recursos AWS criados (todos us-east-1, conta 767397935861)

Recurso	Nome / identificador	Observações
ECR repo	halalosphere-verify-web	imagem do container (nginx + SPA)
CodeBuild	halalosphere-verify-web-build	buildspec deploy/codebuild/buildspec-verify.yml (repo halalosphere-frontend); modo privilegiado (docker)
CodePipeline	halalosphere-verify-web-pipeline	Source(GitHub release) → Build → Deploy ECS

Log group	/aws/ecs/halalsphere-verify-web	logs das tasks
Task definition	halalsphere-verify-web	Fargate, 256 CPU / 512 MB, X86_64; container halalsphere-verify-web porta 3000
Target group	halalsphere-verify-web-tg	tipo IP , HTTP:3000, health check /verify/health (200)
ECS service	halalsphere-verify-web	cluster ecohalal-cluster-fambrashalal-web, 2 tasks, FARGATE_SPOT+FARGATE
Regra de listener	prio 10 em HTTPS:443	path /verify/* → TG acima

Reuso (não criamos novos):

- **Execution role:** fambrashalal-cert-web_production_task_execution_role (a mesma do fambrashalal-cert-web).
- **Security group:** sg-013a70ccd4c44e4b8 (o SG da ALB; mesma config do fambrashalal-cert-web).
- **Subnets:** subnet-0f8c9eac25e08b0a4 , subnet-0d36ec0075e79e5fc (VPC vpc-0d47cab0d440d5de5), **IP público ON** (mesmo padrão do cert-web; pull de imagem via internet).
- **Cert TLS:** wildcard *.fambrashalal.com.br já instalado na ALB (sem ACM novo).

Por que porta 3000 e SG compartilhado: espelha o service fambrashalal-cert-web . A porta 3000 já é liberada pela ALB nesse SG, e a task **não fica exposta na internet** numa porta aberta — só a ALB alcança. Por isso **não foi criado SG novo**.

4. Roteamento na ALB — a única alteração em recurso existente

Listener **HTTPS:443** da ecohalal-fambrashalal-web (...:listener/app/ecohalal-fambrashalal-web/f1a8105ebba087bb/37771d8ee1783b51):

Prioridade	Condição	Ação
10	path /verify/*	forward → halalsphere-verify-web-tg
default	(qualquer outra)	forward → fambrashalal-cert-web-tg (SysHalal)

Garantia de não-disrupção do SysHalal: regra avaliada por prioridade; a primeira que casa vence; senão, default. /certificadovalidate/* **não casa** /verify/* → cai na default → SysHalal, idêntico a antes. A regra é **aditiva e disjunta**. Adicionar regra de listener é operação online (não reinicia listener, não dropa conexões). **Reversível:** deletar a regra restaura o estado anterior.

5. Container / nginx

Imagem (multi-stage, Dockerfile.verify no halalsphere-frontend):

- **build:** node:20-alpine (do **ECR Public**, p/ evitar rate limit do Docker Hub no CodeBuild) → vite build --base=/verify/ + VITE_API_URL=/verify/api .
- **runtime:** nginx:1.27-alpine (ECR Public), escuta **:3000**.

nginx (template com envsubst de \${GC_API_UPSTREAM}):

- `/verify/health` → 200 (health check do TG).
- `/verify/api/*` → `proxy_pass` para `GC_API_UPSTREAM` (= `https://gestaodecertificacoes-api.ecohalal.solutions`), **server-side**.
- `/verify/*` → estático da SPA + fallback SPA p/ `/verify/index.html`.

Env do container (na task def): `GC_API_UPSTREAM=https://gestaodecertificacoes-api.ecohalal.solutions`.

6. CI/CD (automação no padrão)

Pipeline `halalsphere-verify-web-pipeline` (V2):

1. **Source:** GitHub (App) `Ecohalal/halalsphere-frontend`, branch **release** (trigger: push).
2. **Build:** CodeBuild `halalsphere-verify-web-build` → builda a imagem, push no ECR, emite `imagedefinitions.json` (container `halalsphere-verify-web`).
3. **Deploy:** ação **Amazon ECS** → cluster `ecohalal-cluster-fambrashalal-web`, service `halalsphere-verify-web` (rolling, reversão automática).

→ **git push na release** ⇒ **build + deploy automáticos**. (O verify mora no repo do frontend; o pipeline dispara em qualquer push na release. Filtro de path é opcional.)

Permissões: o role do CodeBuild tem `AmazonEC2ContainerRegistryPowerUser` (push ECR); o role do CodePipeline tem as permissões padrão de deploy ECS.

7. Operação

- **Deploy de nova versão:** push na `release` (automático). Manual: rodar o pipeline ou `Update service` com nova revisão da task def.
 - **Health:** GET `https://cert.fambrashalal.com.br/verify/health` → ok. No TG `halalsphere-verify-web-tg` os 2 alvos devem estar **healthy**.
 - **Logs:** CloudWatch `/aws/ecs/halalsphere-verify-web`.
 - **Rollback (desligar o /verify sem afetar o SysHalal):** deletar a regra prio 10 do listener 443 (EC2 → Load Balancers → `ecohalal-fambrashalal-web` → HTTPS:443). Restaura o estado anterior em segundos.
-

8. Backend GC (faz o QR apontar pra cá)

A URL embutida no QR é parametrizada por env no backend GC:

- Serviço `halalsphere-api` (cluster `ecohalal-cluster-main`) — task def com env `QR_VERIFICATION_BASE_URL=https://cert.fambrashalal.com.br/verify`.
 - O deploy do backend é image-only (`imagedefinitions.json`), então a env **persiste** entre deploys de código. Só **certificados novos** nascem com a URL nova (PDF/QR é imutável após a 1ª geração).
-

9. Pontos de atenção

1. **Não editar** a ação **default** do listener 443 nem o certificado — é o que protege o SysHalal.
2. A regra deve permanecer **específica** (`/verify/*`); nada genérico que capture outros paths.
3. Tasks em subnets "private" com **IP público ON** (espelha o cert-web; pull de imagem via internet — não há NAT dedicado para essas tasks).
4. Imagens base vêm do **ECR Public** (Docker Hub anônimo estoura rate limit no CodeBuild).
5. Health check do TG é `/verify/health` (200) — servido pelo próprio nginx, independe da API GC.